김진건

Frontend Engineer · jingeon27@gmail.com · GitHub · 기술 블로그

병역 · 산업기능요원 현역 복무 중(2025.08 –), 복무 1년 경과로 현역 전직 요건 충족

소개

병원이 프로그램을 등록하고 운영팀이 검수한 뒤 사용자가 예약하기까지, 세 제품에 걸친 흐름을 만들었어요. 외부 사이트에서 정보를 보고 전화로 문의하던 건강검진도 복지몰 안에서 신청·결제할 수 있게 옮겼어요. 공급자와 이용자 양쪽의 일이 실제로 이어지는 지점을 보는 프론트엔드 엔지니어예요.

사내 유일한 프론트엔드로 다섯 제품을 운영하며 React 19·Next.js 16 업그레이드도 맡았어요. 테스트 체계와 lint 규칙, 코드 작성 기준을 마련해 AI로 빠르게 작업하면서도 코드 품질과 사용성 문제를 꼼꼼히 확인해요. 코드를 바꿀 때는 기존 설계의 맥락과 기능이 풀어야 할 비즈니스 문제를 함께 살펴요.

주요 경험

- **병원·운영팀·사용자 흐름:** 병원 관리자의 프로그램 등록·검수 요청 화면과 Back-Office 프로그램 검수 화면을 처음 만들었고, 복지몰 예약까지 같이 맡았어요. 임시 진료일이 새로 생겼을 때는 새로 연 날이 예약 불가로 보이지 않도록 세 제품을 함께 맞췄어요.
- **결제:** 건강검진 예약금 결제에 토스페이먼츠 결제창을 붙이면서, 결제창을 오가도 신청 내용이 유지되게 하고 결제 실패·승인 실패와 연속 클릭으로 생기는 중복 주문까지 처리했어요.
- **인앱 WebView:** 앱에서 열리지 않던 `tel:` 전화상담 버튼을 기존 브릿지로 연결해 앱 재배포 없이 고쳤고, 외부 예약 사이트 이동은 네이티브 뒤로 가기 대신 웹 라우터로 돌아오게 해 WebView째 닫히지 않게 했어요.
- **코딩 에이전트 workflow:** 에이전트가 만든 코드는 전부 읽어요. 같은 지적이 반복되면 스킬이나 lint 규칙으로 옮겼고, 복지몰에서는 슬라이스 내부를 직접 import하던 경고 271건을 정리한 뒤 error로 막았어요.
- **신규 서비스 PoC:** PO가 전달한 프로토타입을 바탕으로, 기존 세 제품을 운영하면서 글로벌 사용자 서비스와 CMS를 3주 만에 배포했어요.
- **계측:** 검색·상세·예약·쿠폰 화면에서 Amplitude 이벤트 140개 이상을 새로 붙이거나 고치며, 상세 진입 뒤 예약·상담 버튼을 눌렀는지와 예약 중 어디서 나갔는지 볼 수 있게 발생 위치와 속성을 화면 동작에 맞췄어요.
- **운영 오류:** 직접 붙인 Sentry에서 운영 사용자 57명 이상에게 75건 난 Bottom Sheet 크래시를 찾아 라이브러리 소스로 원인을 좁혔어요. 9월 10일 수정 배포 뒤 9월 28일까지 같은 오류 그룹은 0건이었어요.

기술

제품 개발:	React, TypeScript, Next.js App Router, 모바일 웹·인앱 WebView, TanStack Query(React Query), React Hook Form, Zod
UI·구조:	Radix UI, Ant Design, shadcn/ui·Base UI, dnd-kit, FSD, pnpm·Turborepo
운영·검증:	Sentry, CloudWatch, Vitest, React Testing Library, MSW, Playwright, ESLint 규칙 개발
AI·계측:	Claude Code·Codex, `AGENTS.md`·스킬 설계, git worktree 병렬 작업, Amplitude 행동 이벤트 구현

경력

세이프닥

Frontend Engineer · 2025.05 – 현재

개발팀이 생긴 지 1년쯤 된 2025년 5월, 예약·리뷰·검색 중심의 복지몰 MVP에 합류했어요. 2026년 1월부터는 사내 프론트엔드가 저 한 명이에요. 4월부터는 프론트엔드 개발과 운영을 모두 혼자 하고 있어요. 7월부터는 복지몰·병원 관리자·Back-Office에 글로벌 사용자 서비스와 CMS가 더해져 다섯 제품의 프론트엔드를 개발·운영하고 있어요. 현재 복지몰의 가입 회원은 약 15만 명이에요. 요구사항의 빈틈과 운영 요청은 7월까지 PM·PO와, 이후에는 개발팀장·부팀장과 확인해 왔어요.

건강검진을 기존 서비스 안으로 옮기며

건강검진은 아임웹에서 정보를 본 뒤 전화로 문의하던 별도 흐름이었어요. 다른 진료과와 달리 기업 계약에 따른 검진 코드를 쓰거나 예약금을 먼저 결제하는 경우가 있고 프로그램 구성도 달라, 기존 비급여 프로그램에 그대로 넣기 어려웠어요. 한 제휴 병원이 검진 예약을 세이프닥 안에서 받길 원했지만, 기존 복지몰과 병원 관리자에는 이 흐름을 다룰 화면이 없었어요.

개발팀장·부팀장과 전달받은 스펙의 예외를 확인하며 Back-Office에는 검사항목·검진 구성 등록 화면을, 병원 관리자에는 검진 병원용 화면과 결제취소 접수를, 복지몰에는 검진 상세와 예약금 결제를 만들었어요. 병원 쪽에서는 예약금 결제를 직접 취소할 수 없어서, 결제된 검진 예약을 취소하면 '결제취소 요청 접수'로 안내하게 했어요.

토스페이먼츠 결제창은 모바일에서 결과를 리다이렉트로만 돌려줘서, 결제창을 다녀오면 페이지가 새로 열리고 메모리 상태가 남지 않아요. 그래서 검사 선택·주소·희망 일시가 결제창을 오가도 유지되도록, 폼 값을 결제 시도의 `orderId`와 함께 `sessionStorage`에 저장하고 같은 시도이면서 현재 스키마를 통과한 값만 한 번 되살리게 했어요.

결제창에서 실패로 돌아오거나 서버가 결제를 실패·환불로 처리하면 신청 화면으로 되돌려 입력을 되살리고 사유를 안내해요. 승인 요청 자체가 실패하면 사유를 알린 뒤 예약 목록으로 보내요. 주문 생성부터 결제창이 열리기까지는 한 mutation으로 묶고, 그동안 신청 버튼을 잠가 연속 클릭으로 주문이 두 번 생기지 않게 했어요. dev API 서버를 쓰는 E2E로 토스페이먼츠 테스트 결제창 이동과, 결제창에서 실패로 돌아올 때의 입력 복원, 승인 요청이 실패할 때 예약 목록으로 가는 경로를 확인했어요. 검진 구성과 예약·결제 화면을 나눈 기준은 건강검진 개발 과정에 정리했어요.

글로벌 사용자 서비스와 CMS PoC

2026년 7월에는 PO가 전달한 시연용 프로토타입의 화면을 모두 반영해 글로벌 사용자 서비스와 별도 CMS를 만들었어요. 프로토타입은 예약을 `localStorage`에 담고 관리 화면도 같은 앱에 두고 있었어요. 저는 프론트엔드 셋업부터 구현·검증·배포까지 맡아 화면에 빠져 있던 상세 상태와 인터랙션을 보완했고, 영어·한국어·아랍어(RTL) URL 체계는 그대로 유지했어요.

백엔드와 병행 개발하기 위해 먼저 MSW 목을 만들었지만, 초기 목은 등록 요청을 모두 성공시켰어요. 로컬에서 빌드한 백엔드에 연결하자 필수값과 운영시간·가격 규칙 때문에 거절되는 입력이 드러났고, 서버 DTO의 검증 규칙을 목에도 반영했어요. CMS의 병원·시술·FAQ·블로그 등록은 이 백엔드와 연결해 확인했어요. 기존 세 제품 운영을 병행하면서 7월 31일 사용자 서비스와 CMS를 3주 만에 PoC로 배포했어요.

병원 등록부터 예약·정산까지

병원 관리자의 프로그램 등록·검수 요청과 Back-Office의 검수·반려·승인 화면은 제가 처음 만들었어요. 운영팀에서 반려 버튼을 눌러도 아무 반응이 없다는 제보가 왔어요. 살펴보니 사유 입력 모달이 제출보다 먼저 닫혔고, 반려에도 폼 전체 검증이 걸려 있었어요. 오래전에 등록된 프로그램은 새 할인율·카테고리 규칙을 통과하지 못해 반려조차 막혔어요. 반려와 검수 중 전환은 현재 값으로 보내고, 내용을 확정하는 승인에만 검증을 남겨 당일 수정했어요. 테스트 체계를 도입한 뒤에는 병원 관리자 등록부터 Back-Office의 반려·승인까지 dev API 서버를 쓰는 E2E로 묶었어요.

임시 진료일도 세 제품이 함께 바뀌어야 했어요. 기존에는 임시 휴무만 있었는데, 병원이 쉬는 요일의 특정 날짜를 진료일로 여는 임시 진료일이 새로 생겼어요. 서버가 두 항목을 같은 배열에 `type`으로 구분해 보내게 됐지만 기존 예약 판정은 이 배열을 모두 휴무로 읽는 구조라, 그대로 두면 새로 연 날이 예약 불가로 보일 수 있었어요. 두 관리 화면에 임시 진료일 입력을 따로 두고, 복지몰의 예약 가능일 판정은 `type`으로 두 항목을 나누되 `type`이 없는 옛 데이터는 그대로 휴무로 읽게 맞췄어요. 9월 10일 배포 이후에는 세 제품이 임시 진료일을 똑같이 예약 가능일로 다뤄요.

예약 폼에서는 React Query 캐시·Zustand·로컬 상태에 나뉘어 있던 가족·날짜 선택 상태를 React Hook Form·Zod로 모았어요. 가족을 추가한 뒤 날짜가 어긋나던 동작을 고쳤고, 예약 대상 선택 컴포넌트는 리팩터링 당시 519줄에서 55줄이 됐어요. 예약 폼 리팩터링에 변경 과정을 적었어요.

수기로 관리하던 정산을 내부 예약 데이터와 연결하는 화면은 초기 구현을 인수해 다듬고 배포 전 운영 요구를 반영했어요. 가격 이력을 추가하자는 요청이 왔을 때는 정산 기준이 프로그램 가격인지 견적 금액인지 개발팀장과 먼저 확인했고, 기준이 견적 금액이라 필요하지 않은 이력 구조는 만들지 않았어요.

모바일에서 막히던 순간들

리뷰 작성 화면에서 시술을 고르는 드롭다운이 모바일에서는 키패드에 가려지고 하나 고를 때마다 닫혀 불편하다는 의견을 PO에게 냈어요. 이를 반영해 기획·디자인이 정한 드롭다운 안 복수 선택과, 한 달 뒤 디자인 시스템에서 바뀐 Bottom Sheet를 모두 제가 리뷰 화면에 적용했어요. 검진 선택 화면에서는 정해진 수만큼 고르면 다음 그룹을 자동으로 열자는 요청이 있었어요. 그대로 만들면 이미 고른 항목을 바꿀 때도 다음 그룹으로 넘어가서, 담당자와 논의해 선택 수를 채운 그룹의 미선택 chip만 비활성화하고 빠져 있던 비활성 상태 디자인도 요청해 반영했어요.

앱 WebView에서는 `tel:` 링크가 동작하지 않던 전화상담 버튼을 기존 브릿지의 외부 링크 열기로 연결했어요. 병원·시술 상세에서 외부 예약 사이트로 내보내는 기능을 붙일 때는 첫 구현이 네이티브 뒤로 가기를 불러 이전 화면 대신 WebView째 닫혔어요. 배포 전에 외부 사이트만 브릿지로 열고 돌아오는 건 웹 라우터로 처리하도록 바꿨어요. 둘 다 앱은 바꾸지 않고 웹 배포만으로 해결했어요.

실험에 참여하며 행동을 기록한 일

검색·프로그램 상세·예약·쿠폰 화면에서 Amplitude 행동 이벤트 140개 이상을 새로 붙이거나 고쳤어요. 상세 진입 뒤 예약·상담 버튼을 눌렀는지, 예약 화면에서는 날짜·시간을 고르다 나갔는지 볼 수 있게 발생 위치와 속성을 화면 동작에 맞췄고, SDK 초기화 전에 발송되는 구조였던 로그인 완료 이벤트도 바로잡았어요. 비즈니스 맥락을 파악하려고 상세 개편 구간의 퍼널도 열어 봤지만 메인 화면 개편이 같은 배포에 섞여 효과를 분리해 읽기는 어려웠어요.

PM이 제안한 전화상담 실험에서는 플로팅 버튼을 하단 내비 버튼으로 옮기고 '빠른 예약 문의' 말풍선과 클릭 계측을 붙였어요. 고객센터 번호를 병원 번호로 오해하는 상황이 생겨 안내를 더했고, 실험 중단 결정 뒤에는 채팅상담으로 되돌렸어요.

신규 가입자의 UTM이 빠질 때는 처음에 프론트엔드에서 이를 저장하는 코드를 stage까지 올렸어요. 기획자와 다시 확인해 보니 원인은 Amplitude 사용자 속성 설정이어서, 코드는 운영 배포 전에 되돌리고 설정으로 해결했어요.

AI가 구현한 코드를 제품에 넣기까지

2026년 4월부터 코딩 에이전트를 본격적으로 쓰고 있고, 생성된 코드는 지금도 모두 읽어요. 반복된 지적과 반례는 저장소별 스킬에 남기고, 코드로 판단할 수 있는 기준은 ESLint 규칙으로 옮겼어요. 관리자 서비스 모노레포의 규칙에는 테스트를 붙였고, 복지몰에서는 그 규칙을 옮겨 와 슬라이스 내부 파일 직접 import 경고 271건을 정리한 뒤 error로 잠갔어요.

plan과 테스트를 먼저 검토하고, 에이전트 구현·리뷰 뒤 변경분 테스트·lint·typecheck를 돌려요. 제가 diff와 로컬 화면을 확인한 뒤 전체 CI를 거쳐 QA에 더블 체크를 요청하고, 문제가 나오면 앞 단계로 돌아가요.

9월에는 검증 규칙이 실제로 오류를 잡는지도 점검했어요. 복지몰의 자체 lint 규칙이 동적 import 등을 놓치는 것을 고치고 테스트를 붙여, 수정 전 규칙에서 8건이 실패하는 것을 확인했어요. 글로벌 서비스는 응답 스키마 위반이 경고로만 남던 것을 테스트에서만 실패하게 바꾸고, 목 데이터를 일부러 틀리게 해 실패를 확인했어요.

제가 diff를 읽고도 놓친 Bottom Sheet의 헤더 스크롤 문제는 본문만 스크롤되도록 고치고 브라우저 레이아웃 회귀 테스트를 추가했어요. 테스트 체계를 다시 만든 과정 · 선언적 코드에 관한 글

운영에서 드러난 문제를 끝까지 확인한 일

의료 서비스라 복지몰에 Sentry를 붙일 때 SDK 소스를 따라가 민감 정보가 실리는 경로를 찾아 막고, 이슈에는 회원 ID만 남겨 영향 사용자 수를 볼 수 있게 했어요. Sentry에서 발견한 Bottom Sheet 드래그 크래시는 수정 배포 전까지 운영에서 57명 이상에게 75건 발생했어요. 라이브러리 소스에서 null ref를 읽는 경로를 찾아 그 경로가 없는 버전으로 올렸고, 9월 10일 배포 뒤 9월 28일 확인까지 같은 오류 그룹은 0건이었어요. Sentry 도입 · 크래시 분석

Next.js 16 전환 뒤에는 URL에 검색 조건이 붙은 관리자 목록에서 검색이 멈췄어요. 개발 서버와 jsdom 테스트는 통과했지만 프로덕션 빌드에서는 확인한 목록 7개 중 6개에서 재현됐어요. 업스트림 라우터 캐시 이슈를 확인하고, 서버가 조회 조건을 읽지 않는 화면에서는 같은 경로의 쿼리 변경을 History API로 처리하도록 바꿨어요. 8월 18일 History API로 옮긴 뒤 9월 28일까지 같은 증상의 재보고는 없었어요. 원인과 수정 과정

dev API 서버에서 회원 조회가 잠깐 늦어지자 로그아웃되는 것을 보고 실패 처리 조건을 확인했어요. 기존에는 5xx·타임아웃도 토큰 만료처럼 취급했어요. 401에서만 로그아웃하고 일시적인 실패는 새로고침으로 복구할 수 있는 에러 화면으로 보내도록 바꿨어요.

왓타임

Frontend Engineer · 2024.01 – 2024.11

역할 React·TypeScript로 ATS 'RoundHR'의 채용 관리 웹·채용 담당자용 웹·랜딩·후보자 수집 Electron 앱 개발 (팀원) · **협업** 프론트엔드·백엔드 팀, 디자이너 · **관련 글** Electron 수집 앱 구조 · query-key-factory 적용

제가 합류한 2024년에는 ATS 'RoundHR'의 시장성을 확인하며 기능을 늘리고 있었어요. 채용 관리 웹의 설정 폼·자동화 규칙, 채용 담당자용 웹의 서류·면접 평가 화면을 개발했어요. 면접 템플릿은 네 단계의 입력과 검증을 `useFunnel`로 나눠 중간에 나가도 이어 쓸 수 있게 했어요.

채용 공고 순서부터 후보자 필터의 중첩 트리, 설정 테이블의 행 정렬까지 DnD를 여러 화면에 적용했어요. 화면마다 따로 다루던 드래그 컨텍스트·핸들·정렬 대상을 dnd-kit 기반 공통 컴포넌트로 나눴어요. 폴더를 클릭하면 선택이 풀리는 문제처럼 정렬과 다른 조작이 맞물리는 부분도 계속 고쳤어요.

후보자 수집 Electron 앱에서는 사이트마다 복제돼 있던 로그인 → 수집 → 중복 확인 → 업로드를 공통 순서와 사이트별 adapter로 나눴어요. 잡플래닛·게임잡을 이 구조에 연결하고, 지원자 한 명의 실패가 전체 수집을 멈추지 않도록 격리했어요. Sentry 오류는 서버 오류·인증 및 조회 실패·그 밖의 API 오류로 구분했어요. 초기 구조는 제가 잡았고 현재 코드는 이후 동료가 다시 작성한 부분이 많아요.

지원동기

세이프닥에서는 병원 관리자·Back-Office·복지몰을 함께 개발하며, 병원의 프로그램 관리부터 운영팀의 검수와 사용자의 예약까지 이어지는 흐름을 맡아 왔어요. 사용자 대부분이 제휴기업을 통해 들어오는 구조라, 서비스 성장에는 제휴 병원과 기업을 늘리는 영업의 역할이 컸어요. 저는 사용자가 필요한 정보를 얻고, 가입과 인증을 거쳐 예약까지 불편 없이 갈 수 있도록 제품을 개선해 왔어요.

앞으로는 제가 개선한 제품이 사용자의 행동과 서비스 성장에 어떤 영향을 주는지 더 직접적으로 확인하며 일하고 싶어요. 레슨/과외 빌딩 로그에서 당근 안에 이미 올라오던 선생님 찾는 글을 바탕으로 서비스를 만들고, 유입 경로와 글 작성 과정을 개선해 간 점에 관심이 갔어요. 당근 레슨/과외는 수요와 공급을 늘리는 데 제품이 차지하는 비중이 더 커 보였어요. 많은 이웃이 이용하는 당근에서 제품을 개선해 선생님과 수강생의 참여를 함께 늘리고 싶어 지원했어요.

병원과 사용자 양쪽의 요구를 구현하며, 요구사항의 빈틈을 채우고 운영 문제를 해결해 온 경험으로 팀에 기여할 수 있어요. 동료들과 함께 지표와 사용자 반응을 살피고 가설을 검증하면서, 다음에 무엇을 개선할지 판단하는 역할까지 넓혀 가고 싶어요.

학력·자격

대덕소프트웨어마이스터고등학교 소프트웨어개발과 | 2021.03 – 2024.01

정보처리기능사 | 2023.04